

Task 4

Intermediate Task: Real-Time E-Commerce Analytics

You will build a streaming pipeline:

JSON files → Spark Structured Streaming → Transformations → Aggregations → JSON + Parquet

1. Input data

Create a JSON file called:

orders.json

Put these records in it:

```
{ "order_id": "ORD1001", "customer_id": 501, "customer_name": "John Doe", "product": "Laptop", "category": "Electronics", "quantity": 1, "price": 1200.0, "city": "Toronto", "timestamp": "2026-08-26T10:00:00" }
{ "order_id": "ORD1002", "customer_id": 502, "customer_name": "Sarah Smith", "product": "Phone", "category": "Electronics", "quantity": 2, "price": 800.0, "city": "Montreal", "timestamp": "2026-08-26T10:02:00" }
{ "order_id": "ORD1003", "customer_id": 503, "customer_name": "Mike Brown", "product": "Desk", "category": "Furniture", "quantity": 1, "price": 450.0, "city": "Toronto", "timestamp": "2026-08-26T10:05:00" }
{ "order_id": "ORD1004", "customer_id": 501, "customer_name": "John Doe", "product": "Mouse", "category": "Electronics", "quantity": 3, "price": 25.0, "city": "Toronto", "timestamp": "2026-08-26T10:07:00" }
{ "order_id": "ORD1005", "customer_id": 504, "customer_name": "Emma Wilson", "product": "Chair", "category": "Furniture", "quantity": 2, "price": 150.0, "city": "Vancouver", "timestamp": "2026-08-26T10:10:00" }
{ "order_id": "ORD1006", "customer_id": 505, "customer_name": "David Lee", "product": "Keyboard", "category": "Electronics", "quantity": 2, "price": 75.0, "city": "Montreal", "timestamp": "2026-08-26T10:12:00" }
{ "order_id": "ORD1007", "customer_id": 503, "customer_name": "Mike Brown", "product": "Monitor", "category": "Electronics", "quantity": 2, "price": 300.0, "city": "Toronto", "timestamp": "2026-08-26T10:15:00" }
{ "order_id": "ORD1008", "customer_id": 506, "customer_name": "Lisa Taylor", "product": "Table", "category": "Furniture", "quantity": 1, "price": 600.0, "city": "Vancouver", "timestamp": "2026-08-26T10:18:00" }
```

2. Your task

Read the JSON file **as a streaming source**.

Do **not** use:

```
spark.read.json(...)
```

Use:

```
spark.readStream
```

3. Define the schema

Create an explicit schema containing:

```
order_id  
customer_id  
customer_name  
product  
category  
quantity  
price  
city  
timestamp
```

Remember that `timestamp` should be a Spark `TimestampType`.

4. Read the streaming data

Read:

```
/data/json/orders.json
```

as a streaming `DataFrame`.

5. Create a new column: `total_amount`

Calculate:

```
total_amount = quantity × price
```

For example:

```
quantity = 2  
price = 800
```

```
total_amount = 1600
```

6. Add a customer type

Create a new column called:

```
customer_type
```

Rules:

```
total_amount >= 1000 → "VIP"  
total_amount >= 500  → "Premium"  
otherwise            → "Regular"
```

Use `when()` / `otherwise()`.

7. Filter invalid orders

Only keep orders where:

```
quantity > 0  
price > 0
```

8. Extract the hour

Create:

```
order_hour
```

from the timestamp.

For example:

```
2026-08-26 10:15:00
```

should produce:

```
10
```

Use Spark functions rather than Python code.

9. Add a watermark

Because this is a streaming application, add:

```
10 minutes
```

of watermarking based on the event timestamp.

Conceptually:

```
withWatermark("timestamp", "10 minutes")
```

10. Perform a window aggregation

Calculate total sales for every:

```
5-minute window
```

Group by:

```
window  
category
```

Calculate:

```
total_sales  
total_quantity  
number_of_orders
```

So your result should look conceptually like:

window	category	total_sales	total_quantity	number_orders
10:00 - 10:05	Electronics	...	...	...
10:05 - 10:10	Furniture	...	...	...
10:10 - 10:15	Electronics	...	...	...

11. Create another aggregation

Create a **city-level streaming aggregation**.

Group by:

```
city
```

Calculate:

```
total_sales  
total_quantity  
number_of_orders  
average_order_value
```

This is a separate streaming DataFrame.

12. Write the output

Write the window aggregation to:

```
/data/sink_sales
```

in:

```
Parquet
```

Use:

```
outputMode("append")
```

and create a checkpoint:

```
/data/sink_sales/checkpoint
```

13. Write the city aggregation

Write the city-level aggregation to:

```
/data/sink_city
```

in:

JSON

with its own checkpoint:

/data/sink_city/checkpoint

14. Trigger

Use a processing trigger of:

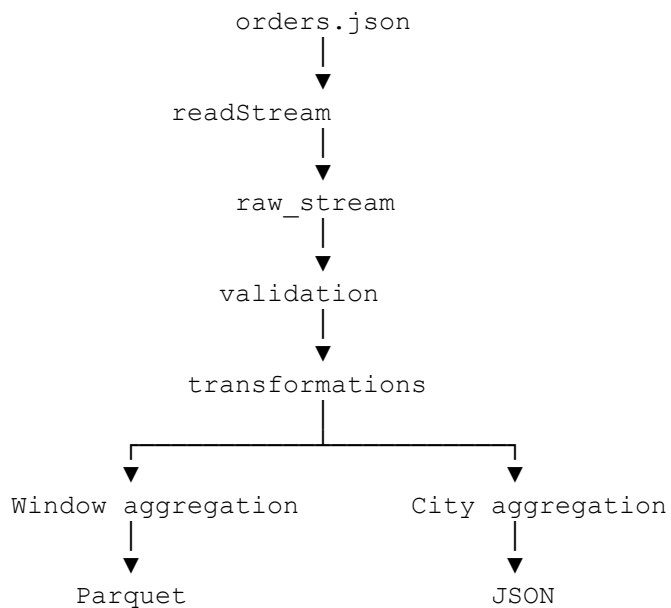
10 seconds

So Spark checks for new data every 10 seconds.

15. Important challenge

Don't put everything into one DataFrame.

Your pipeline should conceptually look like:





What you should practice

This task covers several important Structured Streaming concepts:

Concept	Practice
<code>readStream</code>	✓
Explicit schema	✓
<code>withColumn()</code>	✓
<code>when()</code> / <code>otherwise()</code>	✓
<code>filter()</code>	✓
<code>withWatermark()</code>	✓
<code>window()</code>	✓
<code>groupBy()</code>	✓
<code>sum()</code>	✓
<code>count()</code>	✓
<code>avg()</code>	✓
Append mode	✓
Processing-time trigger	✓
Checkpoint	✓
Parquet sink	✓
JSON sink	✓
Multiple streaming queries	✓